

Lesson 1 — One File Becomes Many

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The one rule

Draw an arrow from each file to every file it needs.

No loops allowed.

```
main.js → game.js → { input.js, entities.js,
render.js } → config.js
```

config.js needs nothing. input.js needs nothing. Those two are the most reusable files you own, and that is *why*.

Option A — script tags (works by double-clicking)

```
<script src="config.js"></script>
<script src="input.js"></script>
<script src="entities.js"></script>
<script src="render.js"></script>
<script src="game.js"></script>
<script src="main.js"></script>
```

Order is forced, not chosen. One global name per file:

```
const Config = { BALL_SPEED: 260 };
```

Option B — modules (needs a local server)

```
// config.js
export const BALL_SPEED = 260;

// game.js
import { BALL_SPEED } from "../config.js";
import * as Render from "../render.js";
```

```
<script type="module" src="main.js">
</script>
```

The `./` and the `.js` are both required.

Modules need a server

Double-clicking gives you:

```
blocked by CORS policy
```

Because every local file is its own origin. Fix:

```
python3 -m http.server 8000
```

Then open `http://localhost:8000`. No internet involved.

What modules give you that script tags do not

	tags	modules
order	by hand	worked out
names	global	private
strict mode	off	always on

A cycle

```
game.js imports hud.js, hud.js imports game.js:
```

```
ReferenceError: Cannot access
'state' before initialization
```

Fix, in order of preference: **pass the value in** · move the shared thing to a third file · admit they are one file.

Words

Coupling — how much A relies on B's insides. Low is good. **Cohesion** — how much one file's contents belong together. High is good. **Entry point** — the single file the page loads. **Topological order** — an order where nothing comes before what it needs.

Section A — Recall

A1. Give two symptoms of a file that has grown too large. Neither may be an error message. [2 marks]

A2. What does an arrow from `game.js` to `config.js` mean, in words? [2 marks]

A3. `input.js` never mentions the ball, the paddle or Breakout. Why is that worth having? [2 marks]

A4. Name three things `type="module"` gives you that six `<script>` tags do not. [3 marks]

A5. Why can a module not be `import` ed from a file opened by double-clicking? Name the fix. [3 marks]

A6. Define *coupling* and *cohesion*, and say which you want high. [2 marks]

Section B — Predict the output

Write your answer down before you run anything.

B1. These tags are in this order. What happens, and what does the console say? [3 marks]

```
<script src="main.js"></script>
<script src="config.js"></script>
```

B2. `audio.js` has `console.log("starting up")` at the top level. Four different modules import it. [3 marks]
How many times does the message print, and why?

B3. What does this print, and what would the same two lines have done in a plain `<script>`? [4 marks]

```
// inside a module
scoer = 10;
console.log("ok");
```

B4. Work out a safe load order for these five files, or explain why there is not one. [4 marks]

```
a.js needs b.js, c.js
b.js needs d.js
c.js needs d.js
d.js needs nothing
e.js needs a.js
```

B5. Now `d.js` also needs `a.js`. What changes about your answer to B4? [3 marks]

Section C — Find and fix the bug

C1. The console says `Failed to resolve module specifier "config.js"`. There is nothing wrong with `config.js` itself. What is wrong? [3 marks]

```
import { BALL_SPEED } from "config.js";
```

C2. `render.js` loads, with no error, and `Config.INK` is `undefined`. Both files are listed in `index.html`. Name two different possible causes. [3 marks]

```
<script src="render.js"></script>
<script src="config.js"></script>
```

C3. This is in `entities.js`. The game runs, but the author has made a design mistake that will cost them later. What is it, and what should they do instead? [4 marks]

```
import { ctx } from "../main.js";

export function drawBall(ball) {
  ctx.beginPath();
  ctx.arc(ball.x, ball.y, ball.r, 0, Math.PI * 2);
  ctx.fill();
}
```

C4. `hud.js` throws `ReferenceError: Cannot access 'state' before initialization`, and nobody has edited `hud.js` for a week. What has happened, and what is the smallest fix? [4 marks]

```
// hud.js
import { state } from "../game.js";
const startingLives = state.lives;
export function drawHud() { /* ... */ }
```

Section D — Build it

Take **your own Breakout** from the beginner capstone — or `code/01-one-big-file.html` if you would rather start from ours — and split it up.

The rule for today: **move code, do not change code**. When you finish, the game must play exactly as it did at the start. If you spot something you want to improve, write it on a sticky note and improve it tomorrow.

- ☐ **Checkpoint 1.** Before touching anything: write the list of files you intend to end up with, and draw the arrows between them. Show somebody. Two minutes here saves twenty later.
- ☐ **Checkpoint 2.** Make `config.js` and move every tuning number into it. Reload. The game still works and your main file is shorter.
- ☐ **Checkpoint 3.** Make `input.js`. When you have finished, search it for the words "ball", "paddle" and "brick". If you find any, the boundary is in the wrong place.
- ☐ **Checkpoint 4.** Make `render.js`. It must be the *only* file containing the text `ctx.` — search the others and prove it.
- ☐ **Checkpoint 5.** Make `entities.js` and `game.js`. The split: entities know *how* to move, game knows *what it means*. A ball falling off the bottom is `entities` reporting and `game` deciding.
- ☐ **Checkpoint 6.** Get `main.js` under fifteen lines. Then play the game for a full minute and compare it against the original, side by side, on two tabs.
- ☐ **Checkpoint 7.** Draw the arrows again, from the real `<script>` or `import` lines. Compare with checkpoint 1. Where you disagreed is where you learned something.

IF YOU HAVE A TERMINAL

Convert your split to modules afterwards: add `export`, add the `import` lines, replace the six tags with one `<script type="module" src="main.js"></script>`, and serve the folder with `python3 -m http.server 8000`. If you do not have a terminal, stop at

checkpoint 7 — you have done the part that matters.</p>

Section E — Design challenge

NO RIGHT ANSWER

Marked on your reasoning, and on whether you name what your idea costs as well as what it gains.</p>

E1. Should `entities.js` know how to *draw* a ball, or should `render.js` know how to draw a ball? Argue both sides, then pick one and say what it costs you.

E2. You are adding a second game mode: the paddle is at the top and the ball falls upwards. Which of your files would you have to edit? If the answer is "most of them", propose better boundaries.

E3. `config.js` holds both `BALL_SPEED` and the level layout. Use the change-together test on that decision. What happens when you have twelve levels?

E4. The hard one. You want to run 10,000 frames of your Breakout with **no browser window at all**, to prove the ball can never escape the walls. Which of your files could run with no canvas, and which could not? What would you move?

Stretch goals

1. Get `main.js` under ten lines.
2. Write a second `render.js` that draws the game with text characters, and switch between them by editing one line.
3. Add mouse control to `input.js` without editing `game.js` at all.
4. Draw your dependency graph, then create a cycle on purpose and watch what the browser says.
5. Attempt E4 for real: run your game rules in Node, with no browser.